

Chapitre 6 : Implémentation

6.1 Technologies utilisées

Frontend

La couche de présentation est développée avec **React 19.2** et **Vite 6.4** comme outil de build. **Tailwind CSS 3.4** assure le stylage. La navigation est gérée par **React Router v7.13**, le cache des données serveur par **TanStack Query v5.90**, et la gestion d'état globale par **Zustand 5.0**.

Backend

Le serveur backend est construit avec **Express.js 5.2** et **TypeScript 5.9**. **Prisma ORM 6.19** abstrait les interactions avec la base de données. L'architecture suit le patron controller / service / repository / policy, garantissant une séparation claire des responsabilités.

Base de données

PostgreSQL est utilisé comme système de gestion de base de données relationnel. Les migrations sont gérées par **Prisma Migrate** avec un pipeline CI/CD dédié. Des index de couverture et des contraintes d'intégrité référentielle assurent la cohérence et les performances.

Catégorie	Technologie	Version
Framework Frontend	React	19.2
Build Tool	Vite	6.4
CSS Framework	Tailwind CSS	3.4
State Management	Zustand + TanStack Query	5.0 / 5.90
Framework Backend	Express.js + TypeScript	5.2 / 5.9
ORM	Prisma	6.19
Base de données	PostgreSQL	Latest
Service IA	FastAPI (Python)	0.135 (3.14)
Monorepo	pnpm + Turborepo	9.15 / 2.8
CI/CD	GitHub Actions	—

Tableau 3 : Stack technologique de FarmConnect

6.2 Développement backend

L'API backend est organisée en modules fonctionnels correspondant chacun à un domaine métier. Les domaines implémentés sont : **auth**, **profile**, **products**, **cart**, **orders**, **payments**, **notifications**, **delivery**, **analytics**, **admin**, **ai**, **favorites**, **coupons** et **reports**.

Chaque module suit une architecture en couches :

- **Controller** : Réception des requêtes HTTP, validation des entrées, délégation à la couche service.
- **Service** : Implémentation de la logique métier et orchestration des opérations.
- **Repository** : Accès aux données via Prisma ORM, abstraction des requêtes de base de données.
- **Policy** : Contrôle d'accès basé sur les rôles (RBAC) et les règles métier.

La sécurité est assurée par plusieurs mécanismes : authentification JWT avec rotation des refresh tokens, middleware de rate limiting, Helmet pour les en-têtes HTTP, validation des entrées avec **Zod**, et logging structuré avec identifiants de requêtes (**Pino**).

6.3 Développement frontend

Le frontend adopte une architecture **feature-first** où chaque fonctionnalité métier est encapsulée dans un module indépendant. Les modules implémentés couvrent : **auth, products, cart, orders, reports, admin, profile, notifications, favorites, reviews** et **coupons**.

Les routes sont protégées par des guards vérifiant le rôle de l'utilisateur connecté. Les providers applicatifs (Auth, Query, Theme, i18n, Notifications) sont encapsulés dans une hiérarchie de contextes React, assurant la disponibilité des données à travers toute l'application.

6.4 Intégration API

Paieement

FarmConnect intègre des méthodes de paiement adaptées au contexte algérien et international : **Cash on Delivery, Baridi Mob, virement bancaire, carte CIB, carte Edahabia** et l'intégration de **Stripe**. Le système implémente un journal d'événements immuable traçant chaque transition : INITIATED → AUTHORIZED → CAPTURED ou FAILED.

Maps et géolocalisation

La géolocalisation est intégrée au niveau des profils producteurs (latitude, longitude) et des adresses (wilaya, commune). Le système supporte le tri des produits par distance, permettant aux acheteurs de trouver les producteurs les plus proches.

Notifications

Le système supporte plusieurs canaux : IN_APP, EMAIL, SMS et PUSH. Les types couvrent l'ensemble du cycle de vie d'une commande (ORDER_PLACED, ORDER_CONFIRMED, ORDER_SHIPPED, ORDER_DELIVERED, ORDER_CANCELLED), les événements de paiement et les alertes producteur (PRODUCT_LOW_STOCK, PRODUCER_VERIFIED).

6.5 Déploiement

Cloud / Docker / CI-CD

L'infrastructure de déploiement repose sur des principes DevOps modernes avec intégration et livraison continues via **GitHub Actions**. Cinq pipelines distincts sont définis :

- **ci.yml** : Validation continue : build, tests, linting pour l'ensemble du monorepo.
- **api-cd.yml** : Déploiement automatique du service API backend.
- **web-cd.yml** : Build et déploiement de l'application frontend.
- **ai-cd.yml** : Déploiement du microservice d'intelligence artificielle.
- **db-migrate.yml** : Exécution automatique des migrations de base de données.

L'application est conçue pour un déploiement cloud avec une architecture containerisée (Docker), assurant la portabilité et la scalabilité horizontale. Les variables d'environnement sont gérées via un fichier **.env.example** documenté pour chaque environnement (développement, staging, production).